Ch1:

Features of Kotlin:

- Concise
- Null safety
- Interoperable
- Smart cast
- Compilation Time
- Tool-friendly
- Extension function

Advantages of Kotlin:

- Easy to learn
- Kotlin is multi-platform

Kotlin for server-side:

- Expressiveness
- Scalability
- Interoperability
- Migration
- Learning Curve
- Kotlin Koans

Variable declaration:

Var

Val

Kotlin Data Type:

1. Number
2. Character
3. Boolean
4. Array
5. String

Array:

1) Using the arrayOf() function
   - val num = arrayOf(1, 2, 3, 4) //implicit type declaration
   - val num = arrayOf(1, 2, 3) //explicit type declaration

2) Using the Array constructor
   - val num = Array(3, {i-> i*1})

Types of String:

1. Escaped String:
   val text1 ="Hello, JavaTpoint"

2. Raw String

val text1 =""" Welcome To kotlin """

ArrayList in Kotlin:

Constructor	Description
ArrayList<E>()	It is used to create an empty ArrayList
ArrayList(capacity: Int)	It is used to create an ArrayList of specified capacity.
ArrayList(elements: Collection<E>)	It is used to create an ArrayList filled from the elements of collection.

Functions of Kotlin ArrayList:

Function	Description
open fun add(element: E): Boolean	It is used to add the specific element into the collection.
open fun add(index: Int, element: E)	It is used to insert an element at specific index.
open fun addAll(elements: Collection<E>): Boolean	It is used to add all the elements in the specified collection to current collection.
open fun addAll(index: Int, elements: Collection<E>): Boolean	It is used to add all the elements of specified collection into the current list at the specified index.
open fun clear()	It is used to removes all elements from the collection.
open fun get(index: Int): E	It is used to return the element at specified index in the list.
open fun indexOf(element: E): Int	It is used to return the index of first occurrence of specified element in the list or return -1 if the specified element in not present in the list.

---

open fun lastIndexOf(element: E): Int	It is used to return the last occurrence of given element from the list or it returns -1 if the given element is not present in the list.
open fun remove(element: E): Boolean	It is used to remove a single instance of the specific element from current collection, if it is available.
open fun removeAt(index: Int): E	It is used to remove the specific index element from the list.
open fun removeRange(startIndex: Int, endIndex: Int)	Its remove the range of elements starting from startIndex to endIndex in which endIndex is not includes.
open fun set(index: Int, element: E): E	It is used to replaces the element from the specified position from current list with the specified element.
open fun toArray(): Array<Any?>	It is used to return new array of type Array<Any?> with the elements of this collection.
open fun toString(): String	It is used to returns a string representation of the object.
fun trimToSize()	It does nothing in this ArrayList implementation.

Kotlin Operators:

1. Arithmetic operator
2. Relation operator
3. Assignment operator
4. Unary operator
5. Logical operator
6. Bitwise operator

•Arithmetic Operators:

Operators	Meaning	Expression	Translate to
+	Addition	a + b	a.plus(b)
–	Subtraction	a – b	a.minus(b)
*	Multiplication	a * b	a.times(b)
/	Division	a / b	a.div(b)
%	Modulus	a % b	a.rem(b)

- **Relational Operators**

Operators	Meaning	Expression	Translate to
>	greater than	a > b	a.compareTo(b) > 0
<	less than	a < b	a.compareTo(b) < 0
>=	greater than or equal to	a >= b	a.compareTo(b) >= 0
<=	less than or equal to	a <= b	a.compareTo(b) <= 0
==	is equal to	a == b	a?.equals(b) ?: (b === null)
!=	not equal to	a != b	!(a?.equals(b) ?: (b === null)) > 0

• Assignment Operators

Operators	Expression	Translate to
+=	a = a + b	a.plusAssign(b) > 0
-=	a = a – b	a.minusAssign(b) < 0
*=	a = a * b	a.timesAssign(b)>= 0
/=	a = a / b	a.divAssign(b) <= 0
%=	a = a % b	a.remAssign(b)

• Unary Operators

Operators	Expression	Translate to
++	++a or a++	a.inc()
—	–a or a–	a.dec()

• Logical Operators

Operators	Meaning	Expression
&&	return true if all expressions are true	(a>b) && (a>c)
||	return true if any of expression is true	(a>b) || (a>c)
!	return complement of the expression	a.not()

- **Bitwise Operators**

Operators	Meaning	Expression
shl	signed shift left	a.shl(b)
shr	signed shift right	a.shr(b)
ushr	unsigned shift right	a.ushr()
and	bitwise and	a.and(b)
or	bitwise or	a.or()
xor	bitwise xor	a.xor()
inv	bitwise inverse	a.inv()

Kotlin Control Flow Statements:

1. Kotlin If...Else Expression
   - if (condition1) {
   - // block of code to be executed if condition1 is true
   - } else if (condition2) {
   - // block of code to be executed if the condition1 is false and condition2 is true
   - } else {
   - // block of code to be executed if the condition1 is false and condition2 is false
   - }
2. Kotlin When Expression

```
val day = 4

val result = when (day) {
  1 -> "Monday"
  2 -> "Tuesday"
  3 -> "Wednesday"
  4 -> "Thursday"
  5 -> "Friday"
  6 -> "Saturday"
  7 -> "Sunday"
  else -> "Invalid day."
}
println(result)
```

Loops:

1. Kotlin for Loop

```
for (item in collection){
//body of loop
}
fun main(args : Array<String>) {
    val marks = arrayOf(80,85,60,90,70)
    for(item in marks){
        println(item)
    }
}
```

2. Kotlin while Loop

```
while(condition){
//body of loop
}
fun main(args: Array<String>){
    var i = 1
    while (i<=5){
        println(i)
        i++
    }
}
```

3. Kotlin Do while Loop

```
fun main(args: Array<String>){
    var i = 1
    do {
        println(i)
        i++
    }
    while (i<=5);
}
```

Kotlin functions

- There are two types of function-

1. Standard library function
2. User-defined function

• Kotlin standard library function

- sqrt()
- print()
- rem()
- toInt()
- readline()
- compareTo()

User defined function:

- Example
- fun mul(num1: Int, num2: Int): Int {
- var number = num1.times(num2)
- return number
- }

•Lambda Expression –

- val lambda_name : Data_type = { argument_List -> code_body }

```
// with type annotation in lambda expression
val sum1 = { a: Int, b: Int -> a + b }

// without type annotation in lambda expression
val sum2:(Int,Int)-> Int  = { a , b -> a + b}

fun main(args: Array<String>) {
    val result1 = sum1(2,3)
    val result2 = sum2(3,4)
    println("The sum of two numbers is: $result1")
    println("The sum of two numbers is: $result2")

    // directly print the return value of lambda
    // without storing in a variable.
    println(sum1(5,7))
}
```

Oop:

class className{ // class header

// property

// member function

}

• var obj1 = className()

Kotlin Nested class and Inner class:

• class outerClass{

• //outer class code

• class nestedClass{

• //nested class code

• }

•}

Inner class:

```
class outerClass{
  //outer class code
   inner class innerClass{
     //nested class code
   }
}
```

- **Types of Kotlin constructors**
- There are two types of constructors in Kotlin:

1. Primary constructor
2. Secondary constructor

1.

```
class myClass(val name: String,var id: Int) {
  // class body
}
```

2.

```
class myClass{

   constructor(id: Int){
      //code
   }
   constructor(name: String, id: Int){
      //code
   }
}
```

Kotlin Visibility Modifier:

• public • protected •internal • private

Types of Inheritance

- There are four types of inheritance in kotlin, those are :
- Single Inheritance
- Multi-level Inheritance
- Multiple Inheritance
- Hybrid Inheritance

Example

```
// parent class
open class Vehicle{

    open fun run(){
        println("Vehicle is running")
    }

    open fun drivenBy(name: String){
        println("Vehicle is driven by: $name")
    }
}
//
```

Example

```
child class
class Jeep: Vehicle(){

    override fun run() {
        println("Driving a Jeep")
    }
    override fun drivenBy(name: String) {
        println("Jeep is driven by $name")
    }
}
fun main() {
    val jeep = Jeep()
    jeep.run()
    jeep.drivenBy("Ninja")
}
```

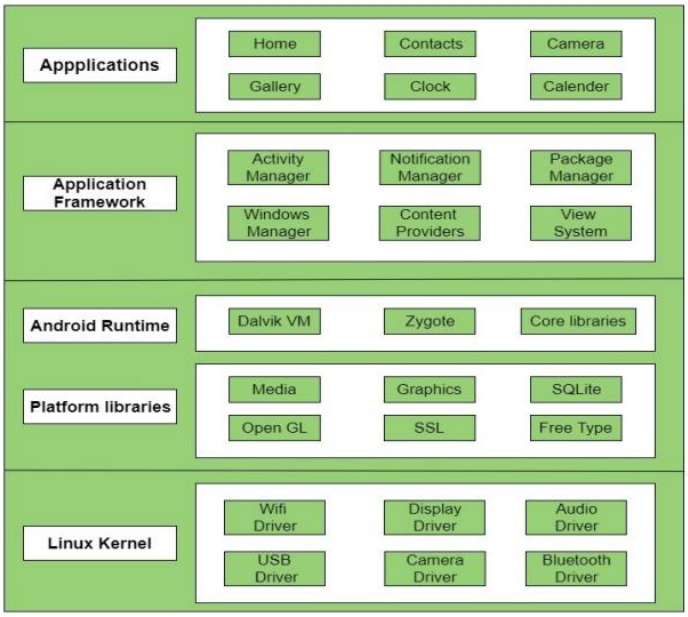
Ch2:

Android Architecture

- The main components of android architecture are following:-

1. Applications
2. Application Framework
3. Android Runtime
4. Platform Libraries
5. Linux Kernel

Android Architecture



• Platform libraries

1. Media
2. Surface manager
3. SGL and OpenGL
4. SQLite
5. Web-Kit
6. SSL

Linux kernel:

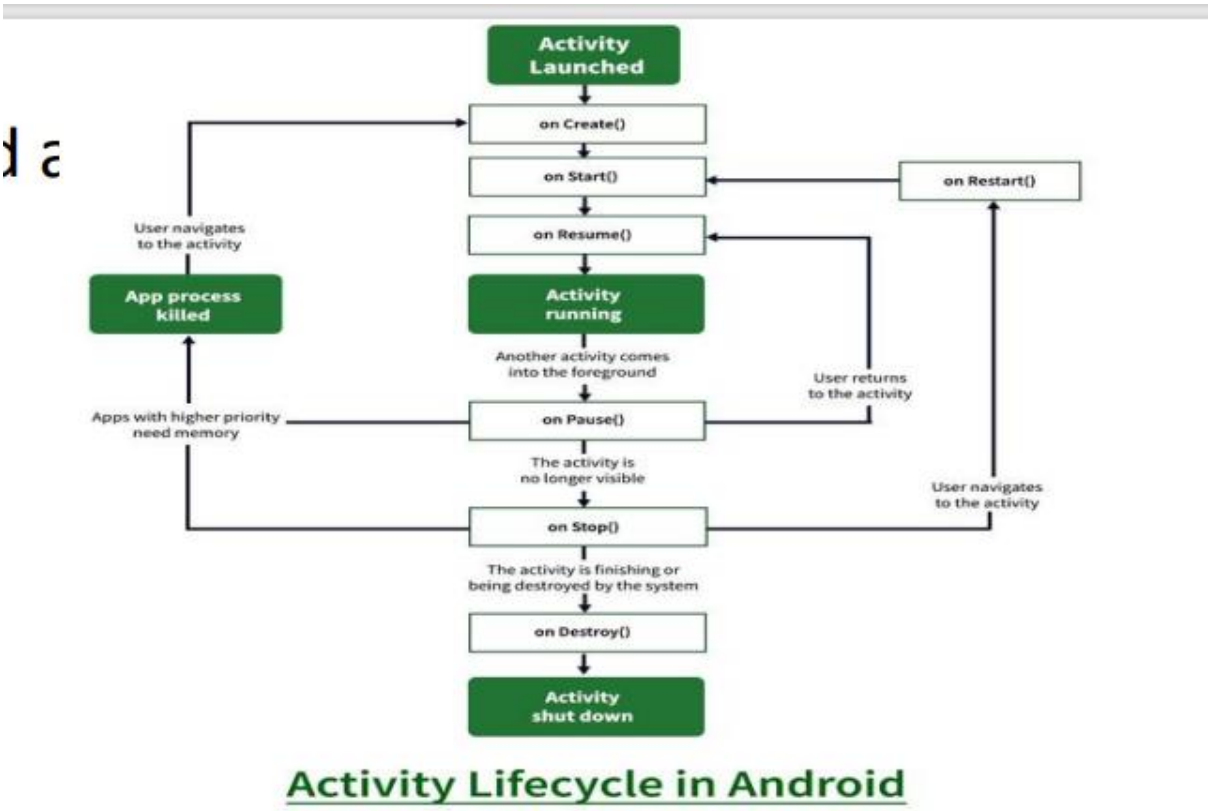
- Security
- Memory Management
- Memory Management
- Process Management
- Network Stack
- Driver Model

Gradle:

- setting.gradle
- build.gradle (project level)
- build.gradle (application level)

Resources in android

Anim
Color
Drawable
Layout
Menu
Raw
Values
Xml



- 1. onCreate()
- 2. onStart()
- 3. onRestart()
- 4. onResume()
- 5. onPause()
- 6. onStop()
- 7. onDestroy()

Android UI Layouts:

1. **LinearLayout**: Aligns child views in a single direction (horizontal or vertical).
2. **RelativeLayout**: Positions child views relative to each other or the parent.
3. **ConstraintLayout**: Positions child views using flexible constraints.
4. **FrameLayout**: Stacks child views on top of each other.
5. **TableLayout**: Arranges child views in rows and columns.
6. **WebView**: Displays web pages within the app.
7. **ListView**: Shows a scrollable list of items in a single column.
8. **GridView**: Displays a scrollable grid of items in rows and columns.

1.liniar layout

android:layout_weight	It is defined individually to the child's views to specify how LinearLayout divides the remaining space amongst the views it contains
android:weightSum	Defines the total weight sum
android:orientation	How the elements should be arranged in the layout. It can be horizontal or vertical.

2.relative

layout_alignParentLeft	It is set "true" to match the left edge of view to the left edge of parent.
layout_alignParentRight	It is set "true" to match the right edge of view to the right edge of parent.
layout_alignParentTop	It is set "true" to match the top edge of view to the top edge of parent.
layout_alignParentBottom	It is set "true" to match the bottom edge of view to the bottom edge of parent.
layout_alignLeft	It accepts another sibling view id and align the view to the left of the specified view id
layout_alignRight	It accepts another sibling view id and align the view to the right of the specified view id.
layout_alignStart	It accepts another sibling view id and align the view to start of the specified view id.
layout_alignEnd	It accepts another sibling view id and align the view to end of specified view id.
layout_centerInParent	When it is set "true", the view will be aligned to the center of parent.
layout_centerHorizontal	When it is set "true", the view will be horizontally centre aligned within its parent.
layout_centerVertical	When it is set "true", the view will be vertically centre aligned within its parent.
layout_toLeftOf	It accepts another sibling view id and places the view left of the specified view id.
layout_toRightOf	It accepts another sibling view id and places the view right of the specified view id.
layout_toStartOf	It accepts another sibling view id and places the view to start of the specified view id.
layout_toEndOf	It accepts another sibling view id and places the view to end of the specified view id.

---

3 Constraint

app:layout_constraintBottom_toBottomOf
app:layout_constraintLeft_toLeftOf
app:layout_constraintRight_toRightOf
app:layout_constraintTop_toTopOf

4.Table

1	**android:id** This is the ID which uniquely identifies the layout.
2	**android:collapseColumns** This specifies the zero-based index of the columns to collapse. The column indices must be separated by a comma: 1, 2, 5.
3	**android:shrinkColumns** The zero-based index of the columns to shrink. The column indices must be separated by a comma: 1, 2, 5.
4	**android:stretchColumns** The zero-based index of the columns to stretch. The column indices must be separated by a comma: 1, 2, 5.

Button in Android:

XML Att
android:id
android:text
android:textColor
android:textSize
android:textStyle
android:textAllCaps
android:background
android:padding
android:visibility
android:gravity

TextView in Android

	android:cursorVisible Makes the cursor visible (the default) or invisible. Default is false.
	android:editable If set to true, specifies that this TextView has an input method.
	android:fontFamily Font family (named by string) for the text.
	android:gravity Specifies how to align the text by the view's x- and/or y-axis when the text is smaller than the view.
	android:hint Hint text to display when the text is empty.

	android:inputType The type of data being placed in a text field. Phone, Date, Time, Number, Password etc.
9	**android:maxHeight** Makes the TextView be at most this many pixels tall.
10	**android:maxWidth** Makes the TextView be at most this many pixels wide.
11	**android:minHeight** Makes the TextView be at least this many pixels tall.
12	**android:minWidth** Makes the TextView be at least this many pixels wide.
13	**android:password** Whether the characters of the field are displayed as password dots instead of themselves. Possible value either "true" or "false".
14	**android:phoneNumber** If set, specifies that this TextView has a phone number input method. Possible value either "true" or "false".

ImageView and ImageButton in Android

Attribute	Description
android:maxHeight	Used to specify a maximum height for this view.
android:maxWidth	Used to specify a maximum width for this view.
android:src	Sets a drawable as the content for this ImageView.
android:scaleType	Controls how the image should be resized or moved to match the size of the ImageView.

CheckBox View in Android

android:id	It is used to uniquely identify the control
android:checked	It is used to specify the current state of checkbox
android:gravity	It is used to specify how to align the text like left, right, center, top, etc.
android:text	It is used to set the text for a checkbox.
android:textColor	It is used to change the color of text.
android:textSize	It is used to specify the size of text.
android:textStyle	It is used to change the style (bold, italic, bolditalic) of text.
android:background	It is used to set the background color for checkbox control.
android:padding	It is used to set the padding from left, right, top and bottom.
android:onClick	It's the name of the method to invoke when the checkbox clicked.
android:visibility	It is used to control the visibility of control.

EditText View in Android

android:inputType	Used to specify what the text entered should be like and for what purpose it will be used. If this is set to none, then the text cannot be edited. Some commonly used constant values for this attribute are: •**text** •**textAutoComplete** - This provides with suggestions as user is typing in text. •**textAutoCorrect** - This will enable auto correct on user input text. •**textPassword** - Display the entered text in form of dots or stars. •**textUri** •**textEmailAddress** •**phone** - This will present only the numeric keyboard to users. •**datetime**, etc.

EditText View in Android

android:minLines	It provides the view, with a height equivalent to the specified number of lines on the screen. So, if you enter a value 2, then by default the EditText view will be 2 lines tall, even without any text added to it. It will the the default height.
android:maxLines	It sets the maximum number of lines that the EditText view can accomodate, **visually**. In other words when we say maxLines has a value 3, it means that the EditText view field will be 3 lines tall after which it will stop increasing in size as more and more text is added to it, usually a scroll bar is shown when number of lines exceeds the limit set.
android:hint	It displays a hint message before anyone types in the Edittext

Android RadioButton

Attribute	Description
android:id	It is used to uniquely identify the control
android:checked	It is used to specify the current state of radio button
android:gravity	It is used to specify how to align the text like left, right, center, top, etc.
android:text	It is used to set the text for the radio button.
android:textColor	It is used to change the color of text.
android:textSize	It is used to specify the size of the text.
android:textStyle	It is used to change the style (bold, italic, bolditalic) of text.
android:background	It is used to set the background color for radio button control.
android:padding	It is used to set the padding from left, right, top and bottom.
android:onClick	It's the name of the method to invoke when the radio button clicked.
android:visibility	It is used to control the visibility of control.

Android ToggleButton

Attribute	Description
android:id	It is used to uniquely identify the control
android:checked	It is used to specify the current state of toggle button
android:gravity	It is used to specify how to align the text like left, right, center, top, etc.
android:text	It is used to set the text.
android:textOn	It is used to set the text when the toggle button is in the ON / Checked state.
android:textOff	It is used to set the text when the toggle button is in the OFF / Unchecked state.
android:textColor	It is used to change the color of text.
android:textSize	It is used to specify the size of text.
android:textStyle	It is used to change the style (bold, italic, bolditalic) of text.
android:background	It is used to set the background color for toggle button control.
android:padding	It is used to set the padding from left, right, top and bottom.
android:drawableBottom	It's drawable to be drawn to the below text.
android:drawableRight	It's drawable to be drawn to the right of the text.
android:drawableLeft	It's drawable to be drawn to the left of text.

Spinner in Android

XML attributes	Description
android:id	Used to specify the id of the view.
android:textAlignment	Used to the text alignment in the dropdown list.
android:background	Used to set the background of the view.
android:padding	Used to set the padding of the view.
android:visibility	Used to set the visibility of the view.
android:gravity	Used to specify the gravity of the view like center, top, bottom, etc

---

DatePicker in Kotlin

XML Attributes	Description
android:id	Used to uniquely identify the control.
android:datePickerMode	Used to specify the mode of datepicker(spinner or calendar)
android:calendarTextColor	Used to specify the color of the text.
android:calendarViewShown	Used to specify whether view of the calendar is shown or not.
android:background	Used to set background color of the Text View.
android:padding	Used to set the padding from left, right, top and bottom.

AutoCompleteTextView in Android

1	**android:completionHint** This defines the hint displayed in the drop down menu.
2	**android:completionHintView** This defines the hint view displayed in the drop down menu.
3	**android:completionThreshold** This defines the number of characters that the user must type before completion suggestions are displayed in a drop down menu.
4	**android:dropDownAnchor** This is the View to anchor the auto-complete dropdown to.
5	**android:dropDownHeight** This specifies the basic height of the dropdown.
6	**android:dropDownHorizontalOffset** The amount of pixels by which the drop down should be offset horizontally.
7	**android:dropDownSelector** This is the selector in a drop down list.
8	**android:dropDownVerticalOffset** The amount of pixels by which the drop down should be offset vertically.
9	**android:dropDownWidth** This specifies the basic width of the dropdown.
10	**android:popupBackground** This sets the background.

Constants of Toast class

Constants	Description
public static final int LENGTH_LONG	displays for a long time
public static final int LENGTH_SHORT	displays for a short time

Methods of Toast class

Methods	Description
public static Toast makeText(Context context, CharSequence text, int duration)	makes the toast message consist of text and time duration
public void show()	displays a toast message
public void setMargin (float horizontalMargin, float verticalMargin)	changes the horizontal and vertical differences

---

ScrollView in Android

Attribute	Description
android:fillViewport	Defines whether the ScrollView should stretch its content to fill the viewport.

Attributes	Description
android:measureAllChildren	Determines whether to measure all children or just those in the VISIBLE or INVISIBLE state when measuring. Defaults to false.

- **Alert Dialog code has three methods:**
- **setTitle()** method for displaying the Alert Dialog box Title
- **setMessage()** method for displaying the message
- **setIcon()** method is used to set the icon on the Alert dialog box.

Android Menu:

1. Popup 2. Contextual 3. Options

Android Listview

Attribute	Description
android:divider	A color or drawable to separate list items.
android:dividerHeight	Divider's height.
android:entries	Reference to an array resource that will populate the ListView.
android:footerDividersEnabled	When set to false, the ListView will not draw the divider before each footer view.
android:headerDividersEnabled	When set to false, the ListView will not draw the divider before each header view.

Intent in Android

- **Types of Android Intents**
- **There are two types of intents in android**

1. Implicit
2. Explicit

Methods	Description
Context.startActivity()	This is to launch a new activity or get an existing activity to be action.
Context.startService()	This is to start a new service or deliver instructions for an existing service.
Context.sendBroadcast()	This is to deliver the message to broadcast receivers.

Tabs and Fragment in android:

- TabLayout:
- ViewPager:
- Fragments:

Ch3:

Shared Preferences:

1. getPreferences()
2. getSharedPreferences()
3. getDefaultSharedPreferences()

- editor.remove("key"):
- editor.clear():

Types of storage:

- App-Specific storage:
- Shared Storage:
- Shared Preferences:
- Databases:

Methods to Store data in External Storage:

- getExternalStoragePublicDirectory():
- getExternalFilesDir(String type):
- getExternalStorageDirectory():

Database - Creation

Sr.No	Method & Description
1	**openDatabase(String path, SQLiteDatabase.CursorFactory factory, int flags, DatabaseErrorHandler errorHandler)** This method only opens the existing database with the appropriate flag mode. The common flags mode could be OPEN_READWRITE OPEN_READONLY
2	**openDatabase(String path, SQLiteDatabase.CursorFactory factory, int flags)** It is similar to the above method as it also opens the existing database but it does not define any handler to handle the errors of databases
3	**openOrCreateDatabase(String path, SQLiteDatabase.CursorFactory factory)** It not only opens but create the database if it not exists. This method is equivalent to openDatabase method.
4	**openOrCreateDatabase(File file, SQLiteDatabase.CursorFactory factory)** This method is similar to above method but it takes the File object as a path rather then a string. It is equivalent to file.getPath()

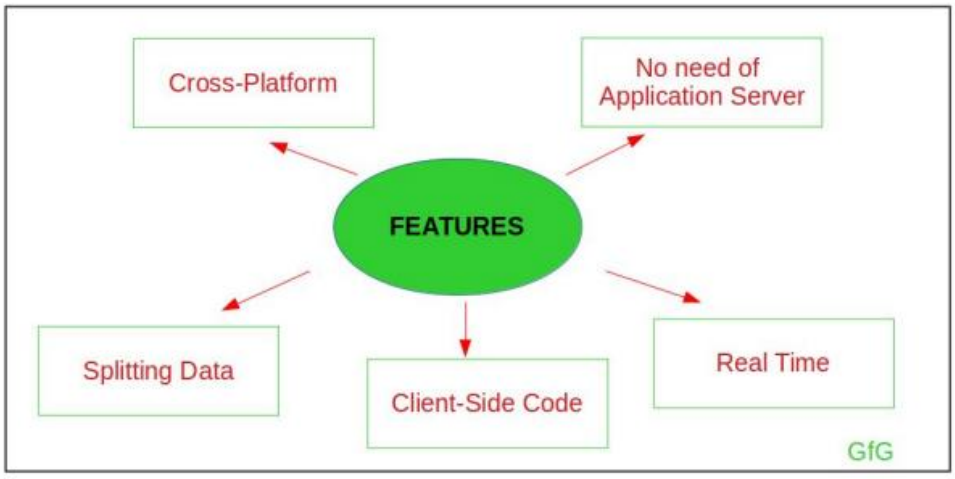
Database – Insertion:

execSQL(String sql, Object[] bindArgs)

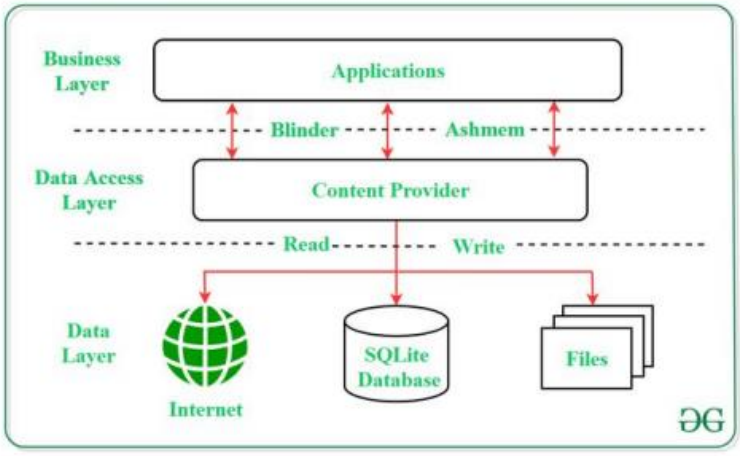
Database - Fetching

Sr.No	Method & Description
1	**getColumnCount()** This method return the total number of columns of the table.
2	**getColumnIndex(String columnName)** This method returns the index number of a column by specifying the name of the column
3	**getColumnName(int columnIndex)** This method returns the name of the column by specifying the index of the column
4	**getColumnNames()** This method returns the array of all the column names of the table.
5	**getCount()** This method returns the total number of rows in the cursor
6	**getPosition()** This method returns the current position of the cursor in the table
7	**isClosed()** This method returns true if the cursor is closed and return false otherwise

Features of Firebase Realtime Database?



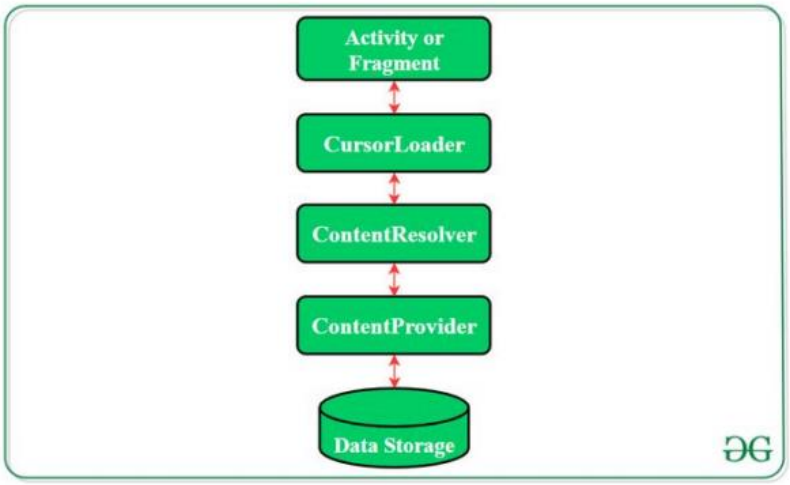
Content Providers



Operations in Content Provider

- Create:
- Read:
- Update:
- Delete:

Working of the Content Provider



six abstract methods of ContentProvider class

Abstract Method	Description
query()	A method that accepts arguments and fetches the data from the desired table. Data is retired as a cursor object.
insert()	To insert a new row in the database of the content provider. It returns the content URI of the inserted row.
update()	This method is used to update the fields of an existing row. It returns the number of rows updated.
delete()	This method is used to delete the existing rows. It returns the number of rows deleted.
getType()	This method returns the Multipurpose Internet Mail Extension(MIME) type of data to the given Content URI.
onCreate()	As the content provider is created, the android system calls this method immediately to initialise the provider.

Ch5:

Animation in Android:

• Property Animation



•View Animation



•Drawable Animation



Important Methods of Animation

Methods	Description
startAnimation()	This method will start the animation.
clearAnimation()	This method will clear the animation running on a specific view.